índice

COMANDOS DE MYSQL

ÍNDICE

crear una base de datos **4**
ver las bases de datos creadas **4**
entrar a una base de datos **4**
borrar una base de datos **4**
mirar las tablas creadas **4**
crear una tabla **4**
ver una tabla **5**
crear tabla temporal **5**
crear una tabla si no existe ya esa tabla **5**
borrar una tabla **5**
MODIFICAR LAS TABLAS **5**
añadir un campo a la tabla después de haberla creado 5
borrar un campo 5
renombrar un campo 5
cambiar el campo y añadir un nuevo tipo 5
modificar el tipo de un campo 6
borrar la clave primaria de una tabla 6
añadir una clave primaria a una tabla 6
ver los atributos de una tabla 6
meter registros **6**
ver los registros creados **7**
borrar registros **7**
actualizar un registro **7**
flecha de relación **7**
restringir que alguien no pueda borrar la clave: **7**
borrar la relación **8**
añadir comentarios **8**
RESUMEN **8**
ORDEN DE EJECUCIÓN **9**
Usos del select **9**
buscar el mayor registro **9**
buscar el menor registro **9**
ver el número de registros **9**
Average / la media de registros **10**
suma de dos registros **10**
like **10**
ejercicios sueltos **10**
subselect **11**
commit; **12**
rollback; **12**
start transaction; / begin; **13**
join **13**
añadir sinónimos a las tablas **14**

vistas **14**
PARTE DE PROGRAMACIÓN **15**
trigger 15
declarar variables 16
procedimientos 17
insertar parámetros 18
declarar variables 18
llamar al procedimiento con parámetros 18
procedimiento con cursor 19
crear un fichero 19
funciones 19

PARA CERRAR LA SHELL (equivale a darle a la cruz de la ventana)
\quit

dentro del shell

\sql
\connect root@localhost
\use MySQL

dentro del mysql

crear una base de datos

create database <nombre_bbdd>;

ver las bases de datos creadas

show databases;

entrar a una base de datos

use <nombre_bbdd>;

borrar una base de datos

(con todo lo que tiene dentro):
drop database <nombre_bbdd>;

DENTRO DE LA BASE DE DATOS:

mirar las tablas creadas

show tables;

crear una tabla

por ejemplo “jugadores”:
create table jugadores (
nif varchar(9) primary key,
nombre varchar(30) not null,
apellidos varchar(50) not null,
dorsal int,
seguro boolean default true,
fec_nac date,
equipo varchar(30));

para crear una tabla con dos PK:
a la hora del create se pone primary key pero con () y dentro los dos atributos.(AL FINAL)

índice

SE PUEDEN CREAR TABLAS TEMPORALES EN LA SHELL QUE CUANDO LA CIERRES SE BORRA

ver una tabla

desc <nombre_tabla>;

crear tabla temporal

create temporary table <nombre_temporal>
(<nombre> <tipo>,
<nombre> <tipo>);

crear una tabla si no existe ya esa tabla

create table if not exists <nombre_tabla> (parametros);

borrar una tabla

(con todo lo que tiene dentro, también sirve para las tablas temporales):
drop table <nombre_tabla>;

MODIFICAR LAS TABLAS

añadir un campo a la tabla después de haberla creado

después de → alter table add column <nombre_campo> <tipo> after <nombre>;
la primera → alter table add column <nombre_campo> <tipo> first;
última →alter table <nombre_tabla> add column <nombre_campo> <tipo>;

ejemplo: alter table paises add column bandera blob;

borrar un campo

alter table <nombre_tabla> drop column <nombre_campo>;

renombrar un campo

alter table <nombre_tabla> rename column <nombre_campo_antiguo> to <n_c_nuevo>;

cambiar el campo y añadir un nuevo tipo

alter table <nombre_tabla> change <campo_antiguo> presidente <tipo>;

modificar el tipo de un campo

alter table <nombre_tabla> modify <nombre_campo> <tipo_nuevo> (opc)<null o not null>;

borrar la clave primaria de una tabla

alter table <nombre_tabla> drop primary key;

añadir una clave primaria a una tabla

alter table <nombre_tabla> add primary key (nombre_campo);

ver los atributos de una tabla

desc <nombre_tabla>; (desc=describir)

(En mysql no hay boolean, él crea un tinyint en el que el 0 es false y cualquier número mayor es true hata 128)

meter registros

con el comando INSERT

PRIMER INSERT → de registro en registro (no sirve para gestionar los defaults)

insert into <nombre_tabla> values (
etc,etc,etc);

SEGUNDO INSERT → eliges las columnas a rellenar y el orden

insert into jugadores (nombre,nif,apellidos,dorsal) values (
etc,etc,etc);

TERCER INSERT → meter más de un registro a la vez

insert into jugadores (nombre,nif,apellidos,dorsal) values (
etc,etc,etc), (etc,etc,etc), (etc,etc,etc);

tiene un problema y es que si intentas meter por ejemplo tres, y tienes cualquier error, no te guarda nada y te sale un error de sintaxis, no lo especifica.

LUIS RECOMIENDA UTILIZAR LA ÚLTIMA SI TENEMOS BIEN CLARO QUE ESTÁ BIEN.

ver los registros creados

SELECT * FROM <nombre_tabla>;
*→ todos los campos
editar el tipo de dato en una tabla (de varchar (30) a varchar (40) no se puede de varchar a boolean ni a int, de int a varchar si, lo cambia a '22')

borrar registros

delete from <nombre_tabla> where marca='LG';

actualizar un registro

update <nombre_tabla> set <campo> = … where <campo> = '...';

añadir campos

Se puede crear una tabla sin PK en la shell, pero en el workbench no.

RELACIONES:

No se puede quitar un campo que ya está en una relación con otra tabla, ni se puede eliminar la tabla.

flecha de relación

ejemplo con la tarea que estamos haciendo:

alter table países
add constraint clave1 - nombre de la relación
foreign key (cod_continente)
references continentes (la que tiene la relación 1) (cod_continente)(la que tiene la n);



restringir que alguien no pueda borrar la clave:

on delete **restrict;**
on update **restrict**;
---
on delete **cascade**;
on update **cascade**;
---
on delete **set null**;
on update **set null**;
esto da error por culpa de la **relación** (el mínimo de la cardinalidad debe ser 0-N para que se pueda usar)
---

borrar la relación

alter table <nombre_tabla> drop constraint <nombre_clave>;

añadir comentarios

de una línea

-- comentario

párrafos

/* comentario
comentario
comentario */

RESUMEN

SENTENCIAS DML (REGISTROS):

- INSERT (añade un registro)
- UPDATE (actualiza un registro)
- DELETE (borra un registro)
- SELECT (selecciona registros)

SENTENCIAS DDL:

- CREATE DATABASE
- CREATE TABLE
- DROP DATABASE
- DROP TABLE

SENTENCIAS DCL:

- GRANT
- REVOKE

Tener en cuenta que cualquier sentencia **DDL no** se puede deshacer, (si eliminas algo que no debias la cagas). **SI** se pueden deshacer las **DML** (insert, delete, update).

índice

ORDEN DE EJECUCIÓN

4 SELECT
1,2 FROM → JOIN
3 WHERE
5 GROUP BY
6 HAVING
7 ORDER BY
8 LIMIT

Usos del select

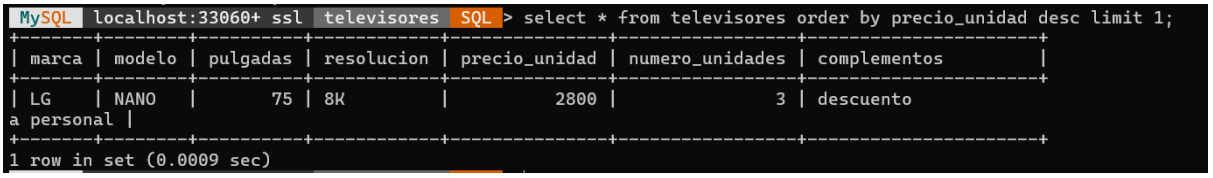
SELECT * FROM <nombre_tabla> → el asterisco selecciona todas las columnas de una tabla.

SELECT <nombre las columnas> FROM <nombre_tabla> ->el select se puede usar de una forma más personalizada usando los nombres de las columnas separándolos con una , (coma). ejem: SELECT marca,modelo,pulgadas FROM televisores

también tenemos otras funciones que podemos usar con select:

buscar el mayor registro

SELECT MAX(precio_unidad) from televisores; (sirve con numero y letras).
equivalencia:



*el 'limit' es importante para paginar.

buscar el menor registro

SELECT MIN(precio_unidad) from televisores;

ver el número de registros

SELECT COUNT(*) from televisores;

COUNT no cuenta campos nulos por eso es importante usar un registro clave, ya que esto asegura el número correcto de registros. Solo falla cuando tiene todos los campos nulos.

Average / la media de registros

SELECT AVG(precio_unidad) from televisores; NÚMERO EXACTO

```
MySQL  localhost:33060+ ssl  actividad01  SQL > SELECT AVG(precio_unidad) from televisores;
+--------------------+
| AVG(precio_unidad) |
+--------------------+
| 1462.2727272727273 |
+--------------------+
1 row in set (0.0015 sec)
```

- con una condición de otro registro:

  select avg(precio_unidad) from televisores where resolucion like '%4k%';
  select avg(precio_unidad) from televisores where resolucion = '4k';

*no usar el like si no se van a usar comodines (%), mejor usar =

suma de dos registros

```
MySQL  localhost:33060+ ssl  actividad01  SQL > SELECT SUM(precio_unidad * numero_unidades) from televisores;
+-------------------------------------+
| SUM(precio_unidad * numero_unidades) |
+-------------------------------------+
|                               66490 |
+-------------------------------------+
1 row in set (0.0048 sec)
```

SELECT SUM(precio_unidad * numero_unidades) from televisores;

like

```
MySQL  localhost:33060+ ssl  televisores  SQL > select * from televisores where complementos like '_r%';
+---------+--------+-----------+------------+---------------+-----------------+------------------+
| marca   | modelo | pulgadas  | resolucion | precio_unidad | numero_unidades | complementos     |
+---------+--------+-----------+------------+---------------+-----------------+------------------+
| LG      | LED    |        65 | 8K         |          1160 |               6 | gratis
altavoces |
| SAMSUNG | AU     |        65 | 4K         |          1200 |               3 | gratis
base     |
| XIAOMI  | QLED   |        75 | 8K         |          2100 |               5 | gratis mando AT  |
+---------+--------+-----------+------------+---------------+-----------------+------------------+
3 rows in set (0.0012 sec)
```

_ = que la primera posición sea cualquiera, luego la r y después cualquiera (%). No tienen mucha utilidad.

ejercicios sueltos

```
MySQL  localhost:33060+ ssl  televisores  SQL > select marca, sum(precio_unidad * numero_unidades) from televisores gro
up by marca;
+---------+-------------------------------------+
| marca   | sum(precio_unidad * numero_unidades) |
+---------+-------------------------------------+
| LG      |                               23040 |
| SAMSUNG |                               18830 |
| SONY    |                               14120 |
| XIAOMI  |                               10500 |
+---------+-------------------------------------+
4 rows in set (0.0012 sec)
MySQL  localhost:33060+ ssl  televisores  SQL > select marca, sum(numero_unidades) from televisores where pulgadas != 5
5 group by marca;
+---------+---------------------+
| marca   | sum(numero_unidades) |
+---------+---------------------+
| LG      |                   9 |
| SAMSUNG |                  17 |
| SONY    |                   3 |
| XIAOMI  |                   5 |
+---------+---------------------+
4 rows in set (0.0008 sec)
```

```
MySQL localhost:33060+ ssl televisores SQL > select marca, sum(numero_unidades) from televisores group by marca;
+---------+----------------------+
| marca   | sum(numero_unidades) |
+---------+----------------------+
| LG      |                   21 |
| SAMSUNG |                   19 |
| SONY    |                   10 |
| XIAOMI  |                    5 |
+---------+----------------------+
4 rows in set (0.0012 sec)
```

```
MySQL localhost:33060+ ssl televisores SQL > select marca, count(*) from televisores where marca != 'samsung' group
by marca;
+--------+----------+
| marca  | count(*) |
+--------+----------+
| LG     |        3 |
| SONY   |        3 |
| XIAOMI |        1 |
+--------+----------+
3 rows in set (0.0422 sec)
```

```
MySQL localhost:33060+ ssl televisores SQL > select marca, count(*) from televisores group by marca having marca !=
'lg';
+---------+----------+
| marca   | count(*) |
+---------+----------+
| SAMSUNG |        4 |
| SONY    |        3 |
| XIAOMI  |        1 |
+---------+----------+
3 rows in set (0.0011 sec)
MySQL localhost:33060+ ssl televisores SQL > select marca, count(*) from televisores group by marca having count(*)
!= 3;
+---------+----------+
| marca   | count(*) |
+---------+----------+
| SAMSUNG |        4 |
| XIAOMI  |        1 |
+---------+----------+
2 rows in set (0.0013 sec)
```

subselect

```
MySQL localhost:33060+ ssl televisores SQL > select * from televisores where precio_unidad = (select max(precio_unid
ad) from televisores);
+---------+--------+----------+------------+---------------+-----------------+----------------------+
| marca   | modelo | pulgadas | resolucion | precio_unidad | numero_unidades | complementos         |
+---------+--------+----------+------------+---------------+-----------------+----------------------+
| LG      | NANO   |       75 | 8K         |          2800 |               3 | descuento
a personal |
| SAMSUNG | AU     |       75 | 8K         |          2800 |               4 | regalo
de ordenador  |
+---------+--------+----------+------------+---------------+-----------------+----------------------+
```

```
MySQL localhost:33060+ ssl televisores SQL > select marca, modelo, sum(precio_unidad * numero_unidades) from televis
ores group by marca, modelo;
+---------+-------------+--------------------------------------+
| marca   | modelo      | sum(precio_unidad * numero_unidades) |
+---------+-------------+--------------------------------------+
| LG      | LED         |                                 6960 |
| LG      | NANO        |                                 8400 |
| LG      | QLED        |                                 7680 |
| SAMSUNG | AU          |                                18050 |
| SAMSUNG | QE          |                                  780 |
| SONY    | BRAVIA OLED |                                 1720 |
| SONY    | QD-OLED     |                                12400 |
| XIAOMI  | QLED        |                                10500 |
+---------+-------------+--------------------------------------+
8 rows in set (0.0013 sec)
MySQL localhost:33060+ ssl televisores SQL > select marca, modelo, sum(precio_unidad * numero_unidades) from televis
ores group by marca, modelo having sum(precio_unidad * numero_unidades) > 10000;
+---------+---------+--------------------------------------+
| marca   | modelo  | sum(precio_unidad * numero_unidades) |
+---------+---------+--------------------------------------+
| SAMSUNG | AU      |                                18050 |
| SONY    | QD-OLED |                                12400 |
| XIAOMI  | QLED    |                                10500 |
+---------+---------+--------------------------------------+
3 rows in set (0.0009 sec)
```

```
MySQL localhost:33060+ ssl televisores SQL > select * from televisores where precio_unidad is null;
+-------+--------+----------+------------+---------------+-----------------+---------------+
| marca | modelo | pulgadas | resolucion | precio_unidad | numero_unidades | complementos  |
+-------+--------+----------+------------+---------------+-----------------+---------------+
| hp    | dcs    |       40 | NULL       |          NULL |            NULL | NULL          |
+-------+--------+----------+------------+---------------+-----------------+---------------+
1 row in set (0.0007 sec)
```

NO ENTRA EN EL EXAMEN

Transacción: ejecutar secuencialmente en un orden específico.

commit;

una vez realizada una acción, se guarda y no se puede volver atrás, por defecto lleva autocommit = 1;

Para poder volver hacia atrás/deshacer cosas, ponemos ‘set autocommit = 0; ‘ pero no se autoguarda, es decir que hay que poner manualmente ‘commit;’ (no se recomienda)

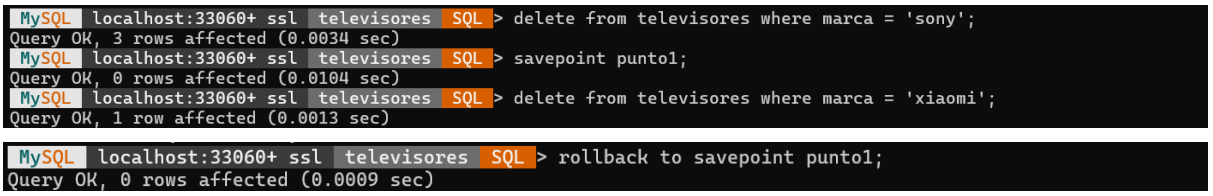
rollback;

Ejecutamos ‘rollback;’ para **deshacer** los cambios realizados en un plazo de 3 horas.

Hay que ir haciendo un commit de vez en cuando para vaciar la zona de memoria que guarda lo realizado posterior al último commit. El rollback funciona igual, esas zonas se llaman segmentos de rollback y al ejecutar el comando se vacía.

Tiene parámetros:

- savepoint: no finaliza la transacción (habría que poner rollback para terminar), vuelve atrás desde el savepoint, eliminando lo editado posteriormente.



start transaction; / begin;

En este modo, se empieza a trabajar en modo transacción, se puede hacer commits y rollback. Acaba cuando se ejecute algún commit o rollback.
Es la manera correcta para no ir cambiando el autocommit 1/0.

Si se cierra la sesión en este modo, SIN haber hecho ni un commit ni un rollback, el sistema hace un rollback, pierdes todos los cambios.

join

es lo mismo que un producto cartesiano (todas las opciones posibles con las opciones disponibles)

```
MySQL localhost:33060+ ssl televisores SQL > select coches.matricula, coches.marca, coches.modelo, propietarios.nombre, propietarios.apellidos from coches;
ERROR: 1054: Unknown column 'propietarios.nombre' in 'field list'
MySQL localhost:33060+ ssl televisores SQL > select coches.matricula, coches.marca, coches.modelo, propietarios.nombre, propietarios.apellidos from coches inner join propietarios;
```

```
MySQL localhost:33060+ ssl televisores SQL > select coches.matricula, coches.marca, coches.modelo, coches.nif_propietario, propietarios.nombre, propietarios.apellidos, propietarios.nif from coches inner join propietarios on coches.nif_propietario = propietarios.nif;
+-----------+-------+---------+-----------------+---------+-----------------+-----------+
| matricula | marca | modelo  | nif_propietario | nombre  | apellidos       | nif       |
+-----------+-------+---------+-----------------+---------+-----------------+-----------+
| 7214MLV   | AUDI  | A4      | 09125712W       | Pedro   | García González | 09125712W |
| 0822SNE   | AUDI  | R8      | 11854321Y       | Mariano | Aparicio Sánchez | 11854321Y |
| 1512MDR   | AUDI  | R8      | 21674298H       | Ana     | Gómez Soto      | 21674298H |
| 3412DSM   | BMW   | X2 M35i | 21674298H       | Ana     | Gómez Soto      | 21674298H |
| 2112TRD   | AUDI  | A4      | 24761425Q       | María   | López Soler     | 24761425Q |
| 2341TRE   | AUDI  | A4      | 45765213C       | Juan    | Martín Gómez    | 45765213C |
| 7287STS   | BMW   | Ix3     | 45765213C       | Juan    | Martín Gómez    | 45765213C |
| 1289DMR   | BMW   | Ix3     | 45897654G       | César   | Santos Blas     | 45897654G |
+-----------+-------+---------+-----------------+---------+-----------------+-----------+
8 rows in set (0.0017 sec)
```

cuando una tabla está a la izquierda, en vez de poner el inner ponemos left. (lo mismo con right)

```
MySQL localhost:33060+ ssl televisores SQL > select coches.matricula, coches.marca, coches.modelo, coches.nif_propietario, propietarios.nombre, propietarios.apellidos, propietarios.nif from coches left join propietarios on coches.nif_propietario = propietarios.nif;
+-----------+-------+---------+-----------------+---------+-----------------+-----------+
| matricula | marca | modelo  | nif_propietario | nombre  | apellidos       | nif       |
+-----------+-------+---------+-----------------+---------+-----------------+-----------+
| 0822SNE   | AUDI  | R8      | 11854321Y       | Mariano | Aparicio Sánchez | 11854321Y |
| 1289DMR   | BMW   | Ix3     | 45897654G       | César   | Santos Blas     | 45897654G |
| 1512MDR   | AUDI  | R8      | 21674298H       | Ana     | Gómez Soto      | 21674298H |
| 2112TRD   | AUDI  | A4      | 24761425Q       | María   | López Soler     | 24761425Q |
| 2341TRE   | AUDI  | A4      | 45765213C       | Juan    | Martín Gómez    | 45765213C |
| 3412DSM   | BMW   | X2 M35i | 21674298H       | Ana     | Gómez Soto      | 21674298H |
| 4498RRE   | BMW   | X2 M35i | NULL            | NULL    | NULL            | NULL      |
| 7214MLV   | AUDI  | A4      | 09125712W       | Pedro   | García González | 09125712W |
| 7287STS   | BMW   | Ix3     | 45765213C       | Juan    | Martín Gómez    | 45765213C |
+-----------+-------+---------+-----------------+---------+-----------------+-----------+
9 rows in set (0.0016 sec)
```

hay una forma más “cutre” en la que no existe el left (con where):

select coches.matricula, coches.marca propietario.nombre propietarios.apellido from coches, propietarios where coches.nif_propietario = propietarios.nif;

añadir sinónimos a las tablas

ejemplo: coches → T1 propietarios → T2

select T2.nombre, T1.matricula, T1.marca, T1.modelo from coches as T1 inner join propietarios as T2 on T1.nif_propietario = T2.nifl;

24/04/26

vistas

Es una ventana que nos permite **ver información** de cierta tabla. Los datos dentro se van actualizando si añades datos en las tablas.

las ventanas se **crean** así con ‘create view V_TVLG as select * from televisores where marca ='lg';
(no se suele usar el asterisco, se suele elegir los registros)
normas → en la misma base de datos no puede haber: dos vistas con el mismo nombre, una vista con el mismo nombre de una tabla.

Las vistas perjudican los inserts, deletes… pero vale la pena por la rapidez en consultar los registros. (además que no va a tardar casi nada, es decir no se nota). A parte, permiten restringir el acceso de la visualización, esto tiene mucho que ver con la seguridad.

PARTE DE PROGRAMACIÓN

como vamos a utilizar el ; más adelante, podemos cambiar el delimitador a otra cosa



trigger

Se usan para generar logs.
Es un código, un **programa que se ejecuta solo cuando ocurre algo**, no se puede ejecutar manualmente. Se ejecutan cuando una persona ejecuta una sentencia DML (insert, delete, update). Cada trigger se asocia solo a una tabla y a un evento/sentencia. No tienen caracteres retroactivos (lo anterior a crear el trigger no hará el código)

Tenemos que tener claro si queremos que se ejecute antes o después de que se inserte el registro (before/after).

ejemplo básico: sacar el importe multiplicando unidades * precio_unidad

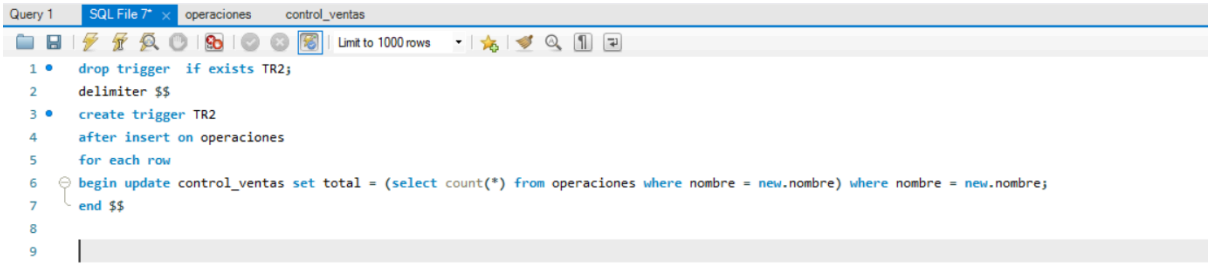
instrucción para primero CREAR el trigger:
cabecera (siempre misma sintaxis), asignar valor a una variable, operación, inicio/final
delimiter $$
create trigger TR1 before insert on operaciones for each row begin set new.importe = new.unidades * new.precio_unidad; end $$

recomendación:

drop trigger if exists TR1;
antes del create trigger para poder ir ejecutando más de una vez, como “actualizarlo” sin tener que eliminarlo y volverlo a crear otra vez.

RECORDAR	
insert	**new.**
delete	**old.**
update	**new. old.**

segundo trigger de ejemplo:

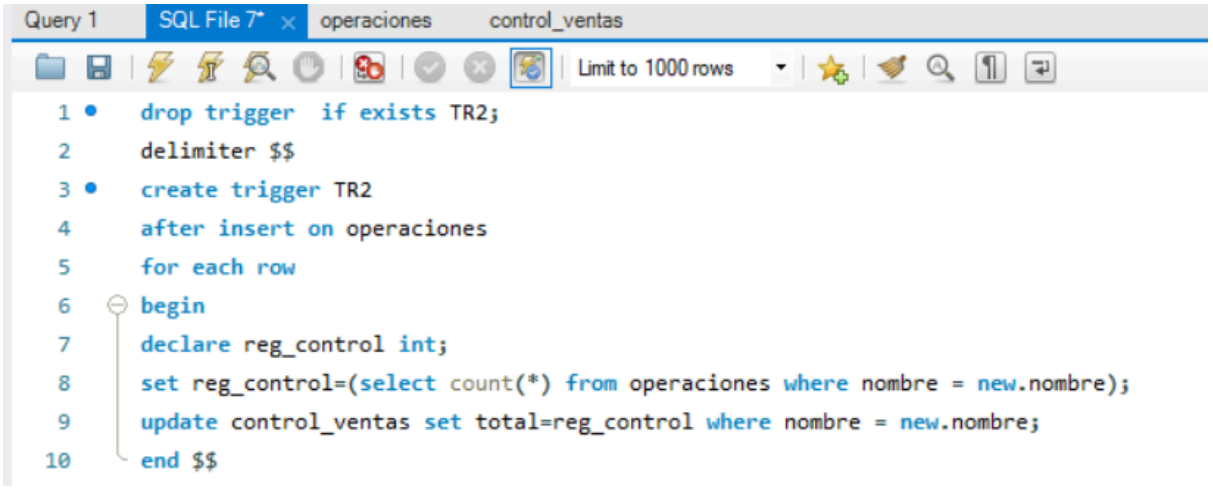


Ahora un poco más avanzado:

declarar variables

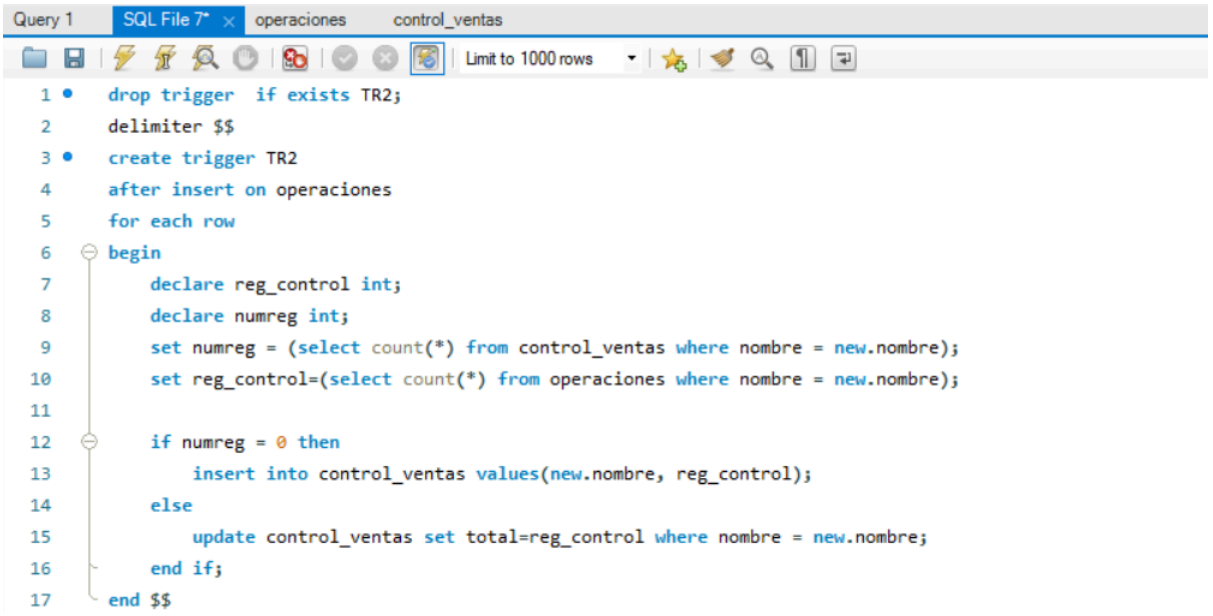
instrucción: declare ej → `declare reg_control int;`

segundo ejemplo mejorado (con la creación de una variable)




```
drop trigger  if exists TR2;
delimiter $$
create trigger TR2
after insert on operaciones
for each row
begin
declare reg_control int;
set reg_control=(select count(*) from operaciones where nombre = new.nombre);
update control_ventas set total=reg_control where nombre = new.nombre;
end $$
```

creando una segunda variable para ver si el nombre de operaciones está en control_ventas y si no está lo añade con un if.




```
drop trigger  if exists TR2;
delimiter $$
create trigger TR2
after insert on operaciones
for each row
begin
    declare reg_control int;
    declare numreg int;
    set numreg = (select count(*) from control_ventas where nombre = new.nombre);
    set reg_control=(select count(*) from operaciones where nombre = new.nombre);

    if numreg = 0 then
        insert into control_ventas values(new.nombre, reg_control);
    else
        update control_ventas set total=reg_control where nombre = new.nombre;
    end if;
end $$
```

Ejercicio planteado para hacerlo nosotros:

“log de respaldo”

crear tabla de log: log_operaciones y luego un trigger que cada vez que se modifica algo en la tabla operaciones va a insertar los datos antiguos en el log_operaciones.

para probarlo cambiar solo los datos unidades, precio unitario y importe

delimiter $$
create trigger TR1
before insert on operaciones
for each row
begin set new.importe = new.unidades * new.precio_unidad;
end $$

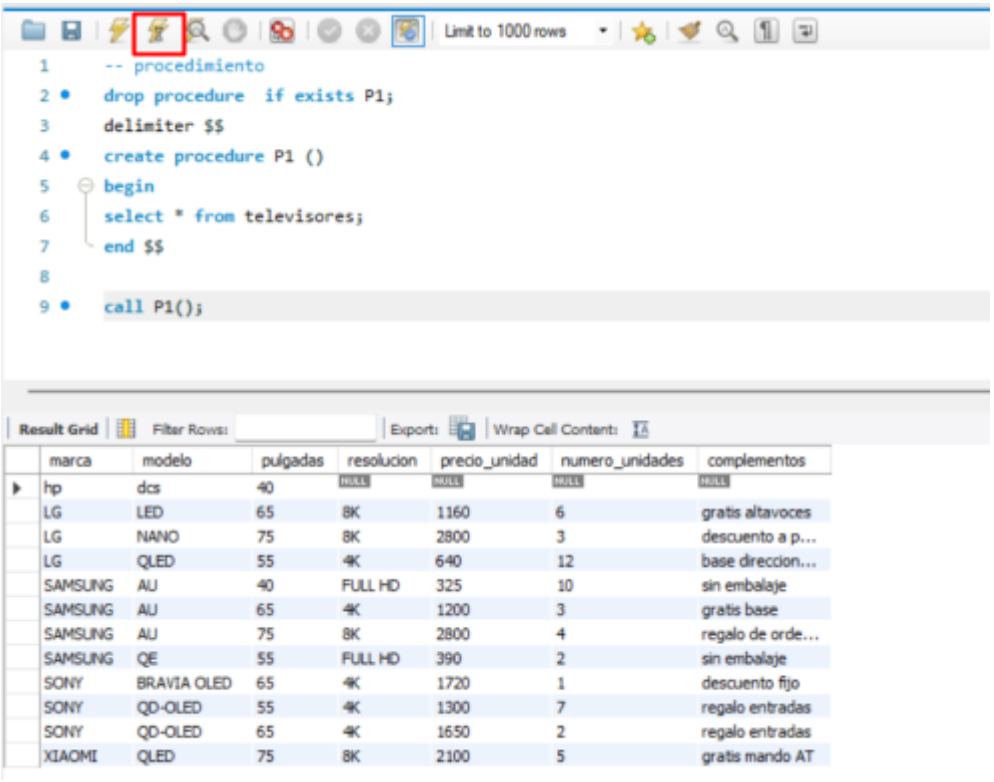
```
1
2   DROP TRIGGER IF EXISTS TR4;
3   DELIMITER $$
4   CREATE TRIGGER TR4
5   BEFORE DELETE ON log_operaciones
6   for each row
7   begin
8       declare primer_id INT;
9       SELECT MIN(nombre) INTO primer_id FROM log_operaciones;
10      IF OLD.nombre = primer_id THEN
11      SIGNAL SQLSTATE '45000'
12      SET MESSAGE_TEXT = 'Acceso denegado: No puedes borrar el registro con el ID más bajo';
13      END IF;
14  END$$
```

procedimientos

Se usan sobre todo para regularizar, añadir o cargar datos. Esta parte ya es programación pura, puede recibir parámetros.

A la hora de crear los procedimientos, son muy parecidos a los triggers; se usa el delimiter, create procedure, begin y end…

para ejecutarlo, se usa call P1(); y le damos al segundo rayo de arriba.



```
1   -- procedimiento
2   drop procedure  if exists P1;
3   delimiter $$
4   create procedure P1 ()
5   begin
6   select * from televisores;
7   end $$
8
9   call P1();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

marca	modelo	pulgadas	resolucion	precio_unidad	numero_unidades	complementos
hp	dcs	40	NULL	NULL	NULL	NULL
LG	LED	65	8K	1160	6	gratis altavoces
LG	NANO	75	8K	2800	3	descuento a p...
LG	QLED	55	4K	640	12	base direccion...
SAMSUNG	AU	40	FULL HD	325	10	sin embalaje
SAMSUNG	AU	65	4K	1200	3	gratis base
SAMSUNG	AU	75	8K	2800	4	regalo de orde...
SAMSUNG	QE	55	FULL HD	390	2	sin embalaje
SONY	BRAVIA OLED	65	4K	1720	1	descuento fijo
SONY	QD-OLED	55	4K	1300	7	regalo entradas
SONY	QD-OLED	65	4K	1650	2	regalo entradas
XIAOMI	QLED	75	8K	2100	5	gratis mando AT

insertar parámetros

dentro de los paréntesis primero la palabra ‘in’ luego el nombre y el tipo.

```
create procedure P1 (in P_marca1 varchar(30), in P_marca2 varchar(30), out P_resultado varchar(200))
```

para luego usarlos en el begin hay dos formas pero a Luis le gusta la de set … =...;

```
set vcaro1=(select max(precio_unidad) from televisores where marca= P_marca1);
set vcaro2=(select max(precio_unidad) from televisores where marca= P_marca2);
```

declarar variables

Las variables se declaran justo después del begin y pueden llevar por **defecto** un parámetro. (w de work por variables de trabajo o v de variables)

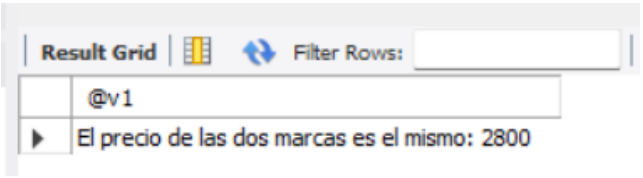
```
declare vcaro1, vcaro2 double;      declare wfin boolean default false;
```

llamar al procedimiento con parámetros

Se añaden los parámetros específicos que se quieren comprobar, ejemplo con el que hemos hecho. Añadiendo una **variable de entorno** (‘@’ al principio)

```
20 •  call P1('LG', 'samsung', @v1);
```

Luego podemos hacer un **select** con la variable de entorno y nos saldrá el resultado.



procedimiento con cursor

Para ello a la hora de declarar el cursor se le añadirá la palabra "**cursor**" después del nombre que le pongamos y un **for** con lo que guardará el cursor.

```
declare cursor1 cursor for select marca, modelo, pulgadas from televisores;
```

Los cursores y manejadores siempre van abajo de todo de las declaraciones, es decir, las variables se declaran antes.
Hay que abrirlos (open cursor1;) y cerrarlos (close cursor1;).

crear un fichero

Se generan en una variable, donde se añade la cabecera y todo lo que queremos que tenga dentro.

luego para que se convierta en un fichero hay que usar el siguiente comando:

```
select wtodo into outfile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/resultado.txt';
```

funciones

A la hora de definir la función, se reciben **parámetros** y se devuelve un dato con **'returns'** obligatoriamente con el tipo de dato y que será, hay seis valores diferentes (son solo explicativas) deterministic, not deterministic, no sql, contains sql, modifies sql data, reads sql data. En general se pone **deterministic**.

este es el primer ejemplo para conocer la estructura

```
1     -- crear una funcion
2 •   drop function if exists F1;
3     delimiter $$
4 •   create function F1(P_importe double) -- p de parametro
5     returns double deterministic
6     begin
7     return (P_importe * 2);
8     end $$
```